

Task 1: Design and Populate a Restaurant Database

Create a structured schema for a food delivery system and populate it with sample data using DDL and DML commands.

- Create a database named foodapp_db and a table named restaurants with columns: restaurant_id (INT, AUTO_INCREMENT, PRIMARY KEY), name (VARCHAR, NOT NULL), city (VARCHAR), cuisine_type (VARCHAR), and rating (DECIMAL(3,1)).
- Insert at least 6 restaurant records spanning at least 3 different cities.
- Update the rating of one restaurant and delete one restaurant record using a WHERE condition.
- Write a SELECT query that retrieves all restaurants sorted by rating in descending order.

```
create database foodapp_db;
```

```
use foodapp_db;
```

```
create table restaurants(  
restaurant_id integer primary key auto_increment,  
name varchar(100) not null,  
city varchar(100),  
cuisine_type varchar(100),  
rating real);
```

```
insert into restaurants(name,city,cuisine_type,rating) values  
( 'Honest', 'Ahemdabad', 'Gujrati', 4.5),  
( 'pasata', 'Rajkot', 'All', 4.7),  
( 'Sankalp', 'Surat', 'South Indian', 4.0),
```

```
('Emperor','Ahemdabad','Fast Food',4.9),  
('The Heritage Palace','Rajkot','Italian',5.0),  
('Royal Kitchen','Keshod','All',4.4);
```

```
update restaurants set rating =4.3 where name ='Sankalp';
```

```
delete from restaurants where name ='Sankalp';
```

```
select * from restaurants order by rating desc;
```

	restaurant_id	name	city	cuisine_type	rating
	8	The Heritage Palace	Rajkot	Italian	5
	7	Emperor	Ahemdabad	Fast Food	4.9
	2	pasata	Rajkot	All	4.7
►	1	Honest	Ahemdabad	Gujrati	4.5
	4	Honest	Ahemdabad	Guirati	4.5

Task 2: Aggregate Revenue Reporting on Orders

Write SELECT queries using aggregate functions and GROUP BY to produce a restaurant-level revenue summary from order data.

- Create an orders table with columns: order_id, restaurant_id, customer_name, total_amount, and order_date.

```
create table orders(  
order_id integer primary key auto_increment,  
restaurant_id integer,
```

customer_name text,

total_amount real,

order_date date);

- **Insert at least 12 orders distributed across at least 4 restaurants.**

**insert into orders(restaurant_id, customer_name, total_amount, order_date)
values**

(1, 'Mital', 1200.00, '2026-10-01'),

(2, 'Nidhi', 1800.00, '2026-10-02'),

(3, 'Teja', 900.00, '2026-10-03'),

(4, 'Riya', 2200.00, '2026-10-04'),

(1, 'Priya', 1500.00, '2026-10-05'),

(2, 'Amit', 2500.00, '2026-10-06'),

(3, 'Neha', 1700.00, '2026-10-07'),

(4, 'Rahul', 3000.00, '2026-10-08'),

(1, 'Kavya', 2000.00, '2026-10-09'),

(2, 'Jay', 1200.00, '2026-10-10'),

(3, 'Hetal', 2600.00, '2026-10-11'),

(4, 'Ankit', 1800.00, '2026-10-12');

order_id	restaurant_id	customer_name	total_amount	order_date
1	1	Mital	1200	2026-10-01
2	2	Nidhi	1800	2026-10-02
3	3	Teja	900	2026-10-03
4	4	Riya	2200	2026-10-04
5	1	Priya	1500	2026-10-05
6	2	Amit	2500	2026-10-06
7	3	Neha	1700	2026-10-07
8	4	Rahul	3000	2026-10-08
9	1	Kavya	2000	2026-10-09
10	2	Jay	1200	2026-10-10
11	3	Hetal	2600	2026-10-11
12	4	Ankit	1800	2026-10-12
NULL	NULL	NULL	NULL	NULL

- Write a query to show total revenue and average order value per restaurant.

```
select restaurant_id,sum(total_amount) as total_revenue,avg(total_amount) as
average_order_value from orders group by restaurant_id;
```

restaurant_id	total_revenue	average_order_value
1	4700	1566.6666666666667
2	5500	1833.3333333333333
3	5200	1733.3333333333333
4	7000	2333.3333333333335

- Write a query using HAVING to list only restaurants whose total revenue exceeds Rs 5,000.

```
select restaurant_id,sum(total_amount) as total_revenue
from orders group by restaurant_id having sum(total_amount) > 5000;
```

	restaurant_id	total_revenue
▶	2	5500
	3	5200
	4	7000

- Write a query to retrieve the 5 most recent orders using **ORDER BY** and **LIMIT**.

select * from orders order by order_date desc limit 5;

	order_id	restaurant_id	customer_name	total_amount	order_date
▶	12	4	Ankit	1800	2026-10-12
	11	3	Hetal	2600	2026-10-11
	10	2	Jay	1200	2026-10-10
	9	1	Kavya	2000	2026-10-09
	8	4	Rahul	3000	2026-10-08
*	NULL	NULL	NULL	NULL	NULL

Task 3: Multi-Table JOINS and Menu Summary View

Build a two-table schema for restaurants and menu items, write JOIN queries to combine data, and create a summary VIEW.

- Create two tables: restaurants (restaurant_id, name, city) and menu_items (item_id, restaurant_id, item_name, price, category).

```
create table menu_items (
item_id integer primary key auto_increment,
restaurant_id integer,
item_name text,
price real,
category text);
```

item_id	restaurant_id	item_name	price	category
1	1	Pav Bhaji	150	Fast Food
2	1	Veg Sandwich	100	Snacks
3	1	Dosa	120	South Indian
4	2	Margherita Pizza	250	Pizza
5	2	Farmhouse Pizza	350	Pizza
6	2	Garlic Bread	130	Sides
7	3	Masala Dosa	140	South Indian
8	3	Idli Sambhar	90	South Indian
9	3	Uttapam	160	South Indian
10	4	Paneer Tikka	220	Starters
11	4	Chole Bhature	180	Main Course
12	4	Dal Makhani	200	Main Course

- Link them with a FOREIGN KEY on restaurant_id. Insert at least 5 restaurants and 12 menu items; at least one restaurant must have no menu items.
- Write an INNER JOIN query listing restaurant name, item name, and price for all available items.

select r.name as restaurant_name,m.item_name,m.price

from restaurants r inner join menu_items m on r.restaurant_id = m.restaurant_id;

restaurant_name	item_name	price
Honest	Pav Bhaji	150
Honest	Veg Sandwich	100
Honest	Dosa	120
pasata	Margherita Pizza	250
pasata	Farmhouse Pizza	350
pasata	Garlic Bread	130
Sankalp	Masala Dosa	140
Sankalp	Idli Sambhar	90
Sankalp	Uttapam	160
Emperor	Paneer Tikka	220
Emperor	Chole Bhature	180
Emperor	Dal Makhani	200

- Write a LEFT JOIN query so that restaurants with no items also appear in results, with NULL values in item columns.

```
select r.name as restaurant_name,m.item_name,m.price,m.category
from restaurants r left join menu_items m on r.restaurant_id =
m.restaurant_id;
```

restaurant_name	item_name	price	category
Honest	Dosa	120	South Indian
Honest	Veg Sandwich	100	Snacks
Honest	Pav Bhaji	150	Fast Food
pasata	Garlic Bread	130	Sides
pasata	Farmhouse Pizza	350	Pizza
pasata	Margherita Pizza	250	Pizza
Sankalp	Uttapam	160	South Indian
Sankalp	Idli Sambhar	90	South Indian
Sankalp	Masala Dosa	140	South Indian
Emperor	Dal Makhani	200	Main Course
Emperor	Chole Bhature	180	Main Course
Emperor	Paneer Tikka	220	Starters
The Heritage Pal...	NULL	NULL	NULL
Royal Kitchen	NULL	NULL	NULL

- Create a VIEW named restaurant_menu_summary showing restaurant name, total number of menu items, and average item price grouped by restaurant.

Task 4: PL/SQL Order Placement Procedure

Write a stored procedure to process a customer order with wallet deduction and full transaction control.

- **Create a stored procedure named place_order that accepts customer_id, restaurant_id, and amount as parameters.**
- **Deduct the amount from the customer's wallet_balance in a customers table and insert a new record into the orders table.**
- **Use a transaction block: if the customer's wallet_balance is insufficient, ROLLBACK everything; otherwise COMMIT.**
- **Test with a valid order and an order exceeding wallet balance. Screenshot both outcomes from MySQL Workbench.**

delimiter //

```
create procedure place_order(
    in p_customer_id integer,
    in p_restaurant_id integer,
    in p_amount real)
begin
    declare v_balance real;

    start transaction;
```

```
-- Lock the customer row and get wallet balance
select wallet_balance into v_balance from customers
where customer_id = p_customer_id for update;

if p_amount <= 0 then
    rollback;
    select 'Invalid order amount' as message;

elseif v_balance >= p_amount THEN

    -- Deduct amount from wallet
    update customers
    set wallet_balance = wallet_balance - p_amount
    where customer_id = p_customer_id;

    -- Insert order details
    insert into orders(customer_id, restaurant_id,
amount)values(p_customer_id, p_restaurant_id, p_amount);
    commit;
    select 'Order placed successfully' as message;
```

```
else
rollback;
select 'Insufficient wallet balance' as message;
end if;
end //
delimiter ;
call place_order(1, 101, 500);
call place_order(2, 101, 999999);
select * from customers;
select * from orders;
```

message
Order placed successfully

message
Insufficient wallet balance